Skill Tree Maker Importer

Overview

The Skill Tree Maker Importer is a Unity asset that allows you to import skill trees created in Skill Tree Maker ↗ directly into Unity.
The system automatically converts the exported .json file into ScriptableObjects, and then builds an interactive skill tree inside a Unity UI Canvas.

Features

- Import skill trees from .json files exported by Skill Tree Maker.
- Automatic conversion into ScriptableObjects.
- Automatic instantiation of nodes and connections inside a Canvas.
- Fully interactive skill tree ready to use in your game.
- Support for skill icons, descriptions, prerequisites, and dependencies.
- Customizable and expandable with your own logic.

System Overview

The importer is divided into two main systems:

1 - Import System (JSON → ScriptableObjects)

- Reads the exported .json file.
- Converts every skill into a SkillNodeData ScriptableObject.
- Automatically organizes nodes into folders inside your Unity project.
- Nodes can be manually edited after import (change icons, descriptions, etc.).

2 - Builder System (ScriptableObjects → Canvas UI)

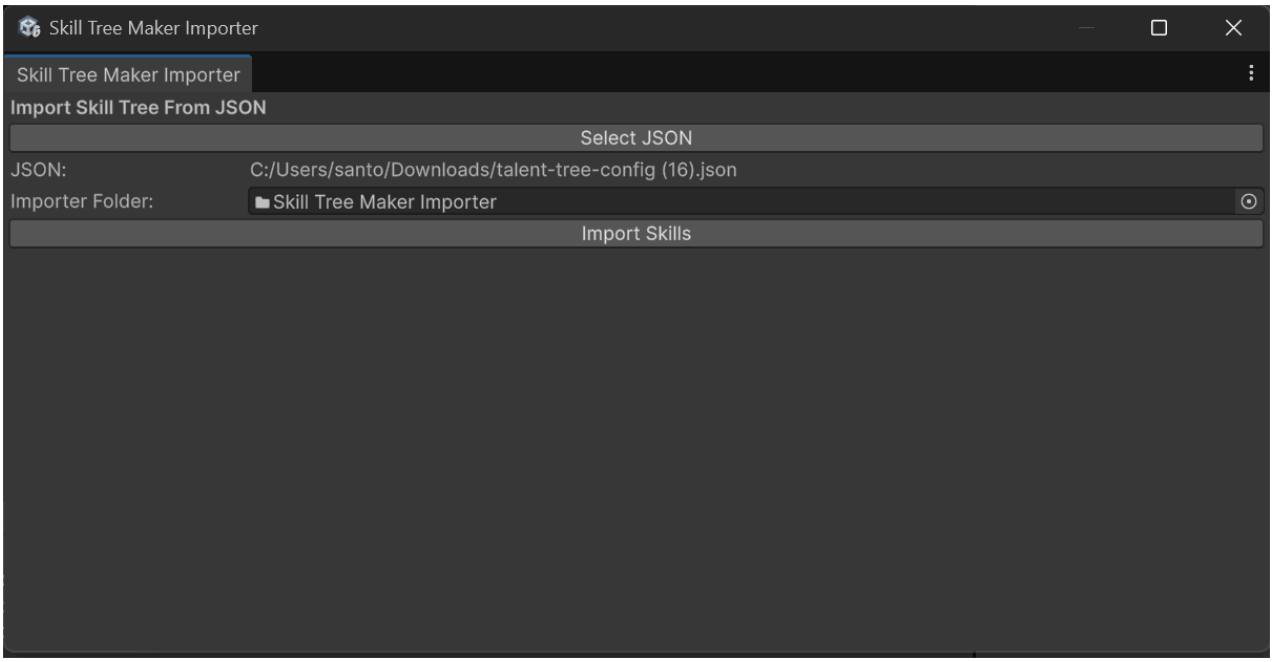
- Reads all imported SkillNodeData.
- Dynamically instantiates UI nodes on a Canvas.
- Connects the nodes with lines representing prerequisites.
- Provides interactivity (selection, tooltips, visual feedback, etc.).

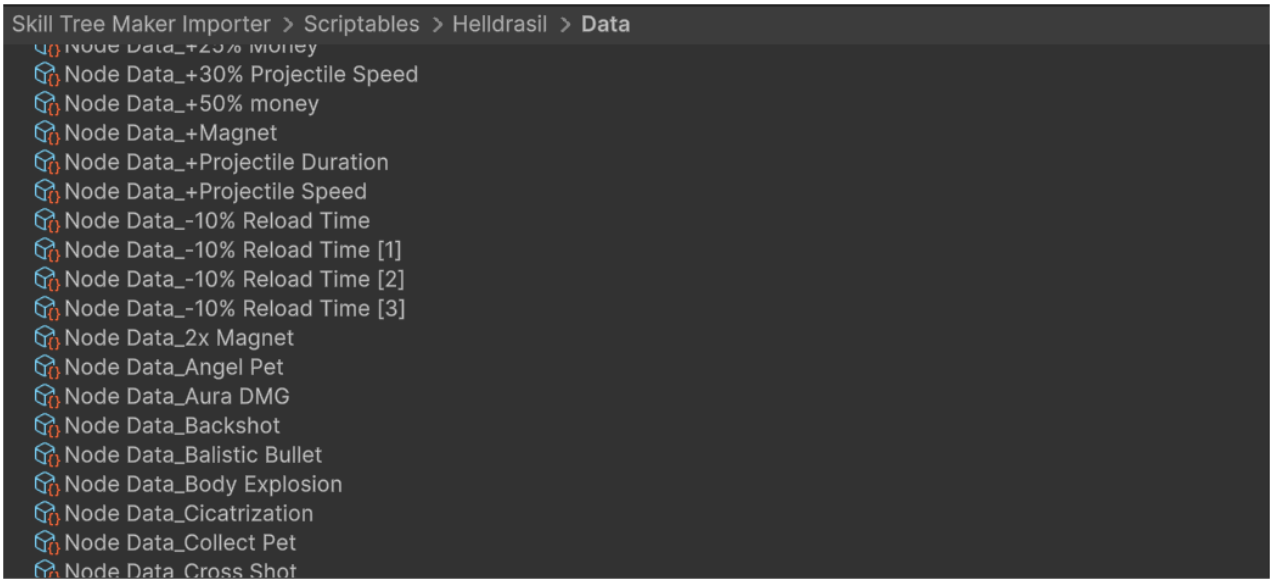
Getting Started

How to Use

1. Export your .json file in the Skill Tree Maker ↗ website
2. In Unity, go to:
   Tools → Skill Tree Maker Importer.
3. Assign the "Skill Tree Maker Importer" folder in the "Importer Folder" field.
4. Select the .json file exported.
5. Click Import Skills.
6. ScriptableObjects will be created inside their respective skill tree folder:

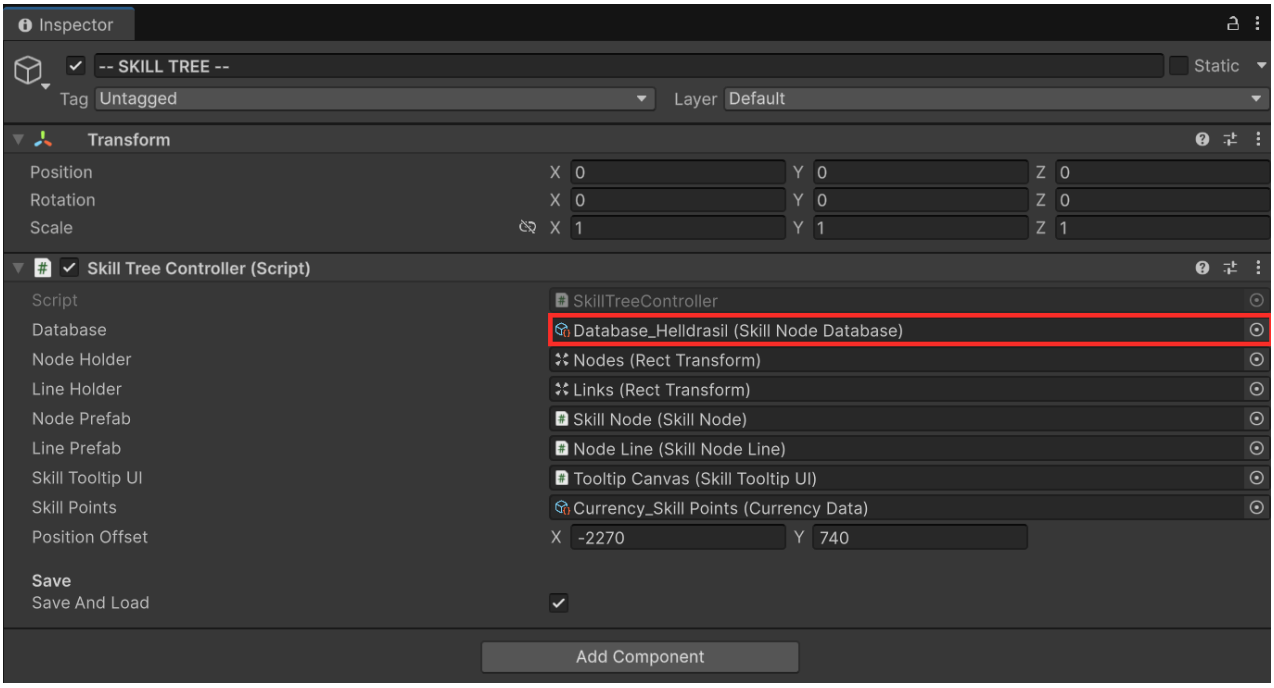
> ⓘ Make sure that your skill tree has given a name, otherwise the import will not work



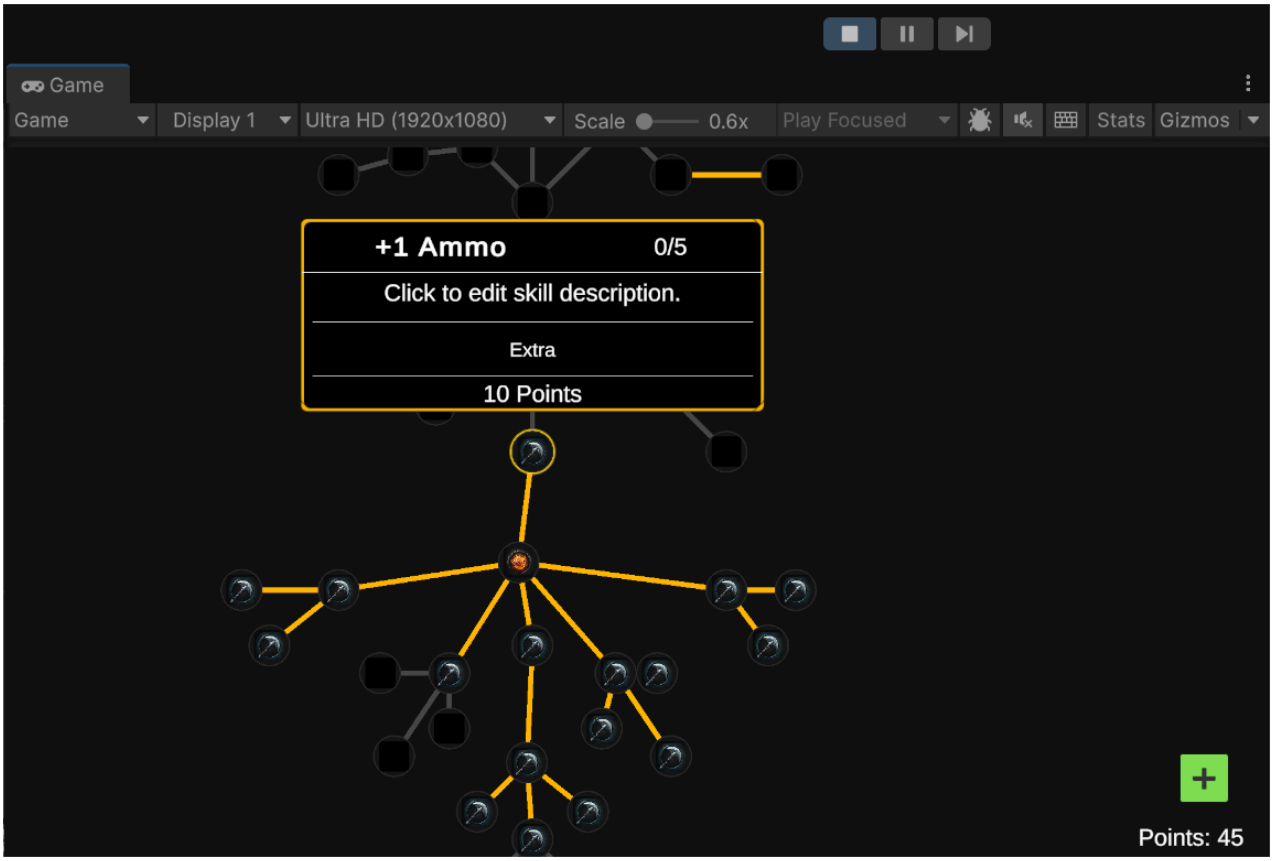


Spawn nodes in the UI Canvas

Now that you converted the json file to scriptable objects, its time to spawn the nodes on the canvas.



1. On your Skill Tree Controller add the database of your recently imported skill tree
2. Press play and have fun!


Game
Game
Display 1
Ultra HD (1920x1080)
Scale
0.6x
Play Focused
Stats
Gizmos
+1 Ammo
0/5
Click to edit skill description.
Extra
10 Points
Points: 45

How to customize

Customization must be done manually.
Below you'll find how to customize your **Skill Nodes** and **Connection Lines**.

Skill Node

To start customizing, you'll need to modify the **Skill Node prefab** located in:
`Skill Tree Maker Importer/Prefabs`

Inside this prefab, you'll find the following objects:

- **Outline**
- **Max Level Outline**
- **Frame** *(The shape of your node)*
  - **Edge**
  - **Frame Mask**

You can freely change the **color** and **sprite** of these components.
Alternatively, you can **remove them** and create your own visual elements.
If you choose to do that, you'll only need to make small adjustments in the code — specifically in the `SkillNodeVisual` script, which contains all the visual logic for the nodes.

In the `SkillNodeVisual` script, you'll also find fields to modify the **colors of the node** in its different states (e.g., locked, unlocked, max level) and the **colors of the connection lines** when they are locked or unlocked.

Skill Node Line

To start customizing, you'll need to modify the **Skill Node Line prefab** located in:
`Skill Tree Maker Importer/Prefabs`

You can adjust the **line width** in the **Line Renderer** component and change the **texture** in the **Line Material** to achieve the desired look for your connection lines.

Tooltip UI

To start customizing, you'll need to modify the **Tooltip Canvas prefab** located in: `Skill Tree Maker Importer/Prefabs`

Main Scripts

Skill Tree Importer

The **SkillTreeImporter** is an **Editor tool** that automatically converts exported **JSON skill tree data** into **ScriptableObjects** inside Unity.
It reads a Skill Tree Maker JSON file and generates all corresponding `SkillNodeData` and `SkillNodeDatabase` assets, ready to be used at runtime.

🧠 Overview

The `SkillTreeImporter` adds a custom window in the Unity Editor under:

```
Tools → Skill Tree Maker Importer
```

From this window, the user can:

1. Select the exported `.json` file from the Skill Tree Maker.
2. Select a target folder where ScriptableObjects will be created.
3. Click **"Import Skills"** to start the import process.

During import, the script:

- Creates all required folder structures (`Skill Trees/<SkillTreeName>/Database` and `Data`).
- Generates one **`SkillNodeData`** **ScriptableObject per skill node**.
- Generates a **`SkillNodeDatabase`** containing all node references.
- Automatically resolves **prerequisites and dependencies** between skill nodes.
- Assigns icons, descriptions, levels, and positions to each node.

⚙️ Key Functions

`ShowWindow()`

Opens the **Skill Tree Maker Importer** window from the Unity Editor menu.
This is the main entry point for starting the import process.

`OnGUI()`

Draws the importer window interface.
Allows the user to:

- Select the JSON file.
- Choose the destination folder.
- Trigger the import process with the **Import Skills** button.

`TryCreateFolders()`

Parses the JSON to retrieve the skill tree name and automatically creates the required folder structure:

- `Skill Trees/<SkillTreeName>/Database`
- `Skill Trees/<SkillTreeName>/Data`

`AssetCreation()`

The **core function** of the importer.
It performs the full JSON-to-asset conversion process:

- Reads all skill data from the JSON file.
- Creates and configures `SkillNodeData` ScriptableObjects.
- Assigns properties such as name, description, icon, position, and level.
- Resolves parent/child node references (prerequisites and dependencies).
- Creates or updates the `SkillNodeDatabase` asset with all nodes included.

Finally, it saves and refreshes the `AssetDatabase`.

`CreateData(SkillNodeData data)`

Creates and saves a new `SkillNodeData` asset on disk if it does not already exist.

`CreateDatabase(List<SkillNodeData> datas)`

Creates or updates a `SkillNodeDatabase` asset containing all imported node data for the current skill tree.

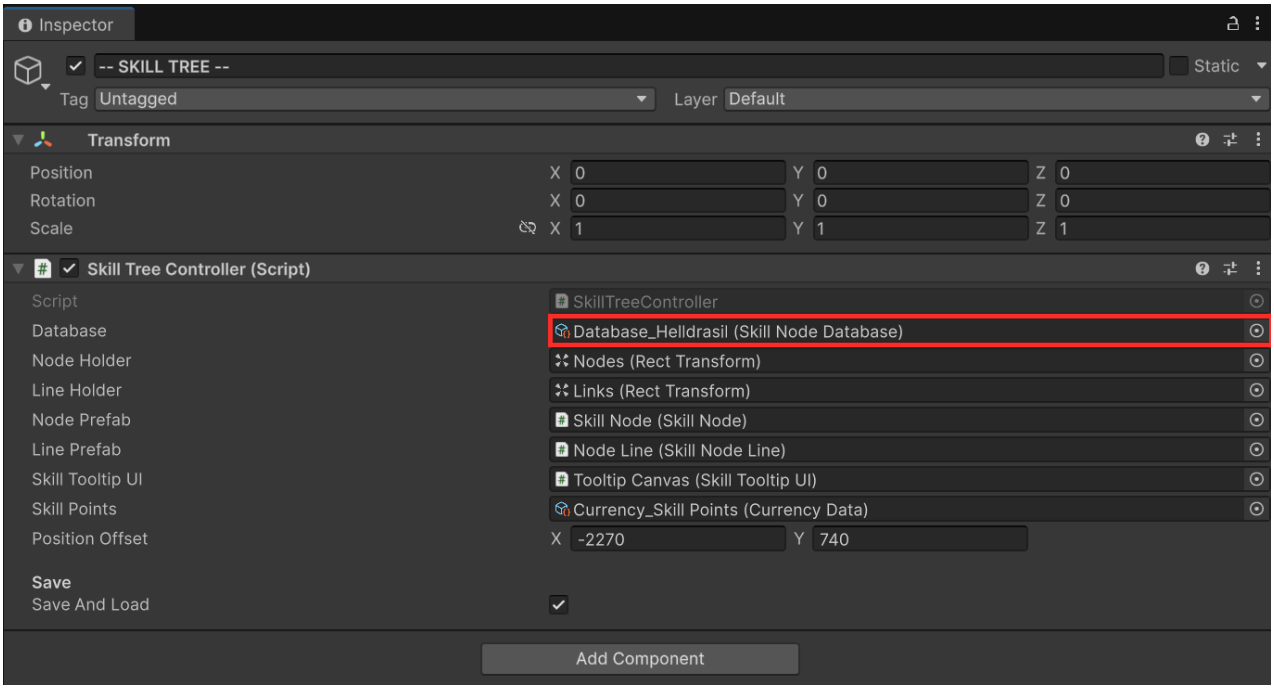
`CreateFolderIfMissing(string path, string folderName, out string fullPath)`

Utility function that checks if a folder exists in the Unity project.
If not, it creates the folder and returns its full path.

Skill Tree Controller

The **SkillTreeController** is responsible for **instantiating and managing all Skill Nodes in the UI Canvas at runtime**.
It loads the `SkillNodeDatabase`, creates all `SkillNode` objects dynamically, connects them visually, and updates their state and interactivity based on the player's data and progression.

This script acts as the **runtime bridge** between the imported ScriptableObjects (`SkillNodeData` and `SkillNodeDatabase`) and the actual skill tree UI that the player interacts with.



🧠 Overview

At runtime, the `SkillTreeController`:

1. Loads the selected `SkillNodeDatabase`.
2. Spawns all nodes (`SkillNode` prefabs) on the Canvas.
3. Connects parent and child nodes using visual lines (`SkillNodeLine`).
4. Initializes each node's data, position, and connections.
5. Handles player interactions such as **selecting**, **deselecting**, or **buying** skills.
6. Optionally saves and reloads the player's skill tree progress automatically.

⚙️ Exposed Options

Field	Type	Description
OnNodeBuyed	`Action`	Event triggered when the player successfully purchases a skill node.
saveAndLoad	`bool`	Enables automatic save/load of the skill tree state. When true, the player's progress is saved and reloaded each session.
tryLoadTimer	`float` *(private)*	Timer used internally to periodically check and attempt reloading the skill tree if the database is changed or not yet loaded.
nodeHolder	`Transform`	The parent container in the Canvas where all `SkillNode` instances are spawned.
lineHolder	`Transform`	The parent container used to hold all visual connection lines (`SkillNodeLine`).
nodePrefab	`SkillNode`	The prefab template used to instantiate each node in the tree.
linePrefab	`SkillNodeLine`	The prefab used to draw visual links between nodes.
database	`SkillNodeDatabase`	The skill tree database containing all node data (imported via the SkillTreeImporter).
skillTooltipUI	`SkillTooltipUI`	The UI element that displays detailed info for the selected skill.
activeSkillNode	`SkillNode` *(private)*	Stores the currently selected skill node.

🔧 Main Methods

`Start()`

Initializes the controller when the scene starts.

- Configures save/load behavior based on the `saveAndLoad` flag.
- Attempts to load the skill tree by calling `TryLoad()`.

`Update()`

Runs every frame.
Increments the internal `tryLoadTimer` and periodically reattempts loading the skill tree if it hasn't been initialized yet.

`TryLoad()`

Checks if a database is assigned and different from the currently active one.
If so, starts the asynchronous loading process through `Load()`.

`IEnumerator Load()`

Coroutine responsible for clearing existing nodes and lines, then calling `SpawnNodes()` to rebuild the skill tree based on the selected database.
Steps performed:

1. Clears existing nodes and connections.
2. Waits a frame to ensure UI cleanup.
3. Calls `SpawnNodes()` to generate new skill nodes.

`SpawnNodes()`

Core function that **instantiates and initializes all skill nodes** in the tree.
It executes in multiple steps:

1. **Clear dictionaries and lists** used to track node-data relationships.
2. **Spawn all nodes** from the `SkillNodeDatabase` using the `nodePrefab`.
3. **Rebuild parent and child relationships** based on data dependencies.
4. **Initialize nodes** with their respective data (`node.Initialize()`).
5. **Create visual lines** between connected nodes (`node.SetupLinks()`).
6. **Update each node's state and visuals** (`node.UpdateInfo()`).

If no database is assigned, it logs an error in the Console.

`SpawnLine()`

Instantiates and returns a new visual connection line (`SkillNodeLine`) under the `lineHolder` transform.

`UpdateInfo()`

Forces an update of the skill tooltip UI, refreshing the display with the latest selected node's data.

`HandleNodeSelection(SkillNode newNodeSelected)`

Called when the player selects a new skill node.

- Deselects the previously active node (if any).
- Stores the newly selected node.
- Displays its information in the tooltip UI.

`HandleNodeDeselection(SkillNode newNodeSelected)`

Clears the active selection and hides the tooltip UI.

Skill Node

The `SkillNode` class represents an individual node within the skill tree system. Each node manages its own state (locked, unlocked, buyable, or maxed), visual updates, and interactions with the player through UI buttons and events.

- Handles initialization and linking between parent and child nodes.
- Manages unlocking and purchasing logic based on skill points and parent nodes.
- Updates its visual state (selection, availability, and maxed-out visuals).
- Saves and loads persistent node data (level, unlock state).
- Communicates with the `SkillTreeController` to synchronize updates across the tree.

Properties

- **`controller`** – Reference to the `SkillTreeController` that manages all nodes.
- **`visual`** – Reference to the `SkillNodeVisual` that controls the node's appearance.
- **`button`** – The UI button used to trigger node actions.
- **`nodeData`** – The `SkillNodeData` asset containing configuration (costs, icon, position, etc.).
- **`parentNodes`** / **`childNodes`** – Lists defining the graph structure of the skill tree.
- **`stateMachine`** – Controls the node's logic state (locked, unlocked, buyable, maxed).
- **`level`** – Current level of the skill node.
- **`isMaxed`** / **`isSelected`** – Status flags used for logic and visuals.
- **`unlockedInfo`** – Helper that tracks unlock requirements and progress.
- **`OnClicked`**, **`OnBuyed`**, **`OnUnlocked`**, **`OnMaxed`**, etc. – Actions and UnityEvents for external callbacks.

Methods

- **`Initialize(SkillTreeController, SkillNodeData)`**
  Sets up references, loads saved data, initializes visuals, connects listeners, and defines the starting state.
- **`UpdateInfo()`**
  Refreshes the node's state and visuals based on current skill points, unlocks, and level.
- **`SetupLinks()`** / **`LinkChild(SkillNode)`**
  Creates line connections between parent and child nodes using `SkillNodeLine`.
- **`TryUnlock()`**
  Checks whether all parent requirements are met and updates the state accordingly.
- **`HandleAvaibility()`**
  Determines if the node can be bought based on skill points and state.
- **`HandleMaxed()`**
  Verifies whether the node has reached its maximum level and updates its state.
- **`TryBuy()`** / **`Buy()`**
  Handles purchasing logic: spends skill points, increases level, triggers events, and saves progress.
- **`SelectMe()`** / **`DeselectMe()`**
  Manages node selection behavior and visual feedback.
- **`Save<T>()`** / **`Load<T>()`**
  Utility methods for saving and retrieving node data persistently through `ProvisorySave`.

Skill Node Line

The `SkillNodeLine` class visually connects two `SkillNode` objects within the skill tree using a `LineRenderer`.
It dynamically updates the line position to reflect UI movement or layout changes in real time.

- Draws and updates a visual connection between two skill nodes.
- Adapts to different canvas rendering modes (`ScreenSpaceOverlay`, `ScreenSpaceCamera`, `WorldSpace`).
- Provides color control for better visual feedback (e.g., unlocked, locked, or selected states).

Properties

- **`line`** – The `LineRenderer` component responsible for drawing the line.
- **`nodeA`** / **`nodeB`** – References to the two connected `SkillNode` instances.
- **`canvas`** – The parent UI canvas; used to adjust line rendering depending on the canvas mode.

Methods

- **`Initialize(SkillNode nodeA, SkillNode nodeB)`**
  Sets the two nodes to connect, finds the canvas in the hierarchy, and performs the initial line update.
- **`Update()`**
  Continuously updates the line's position every frame to ensure it follows both nodes correctly.
- **`UpdateLine()`**
  Calculates the correct positions for the `LineRenderer` based on the canvas's `RenderMode`:
  - In **`ScreenSpaceOverlay`**, it converts node positions from screen to world coordinates.
  - In **`ScreenSpaceCamera`**, it uses the node's world positions directly.
  - In **`WorldSpace`**, the line is not adjusted (reserved for 3D use cases).
- **`SetColor(Color color)`**
  Changes both the start and end color of the line to visually indicate state (e.g., active, inactive).

Skill Node Visual

The `SkillNodeVisual` class handles all **visual aspects** of a single `SkillNode` in the UI.
It controls icons, outlines, colors, and visual states to clearly represent the node's current status (locked, unlocked, buyable, or maxed out).

- Visually represents the state of a skill node in the UI.
- Updates colors, outlines, and highlights based on the node's logic.
- Changes the color of connecting lines when the node becomes unlocked or upgraded.
- Provides easy access to the node's button and icon image for other systems.

Properties

- **`iconImage`** – The main `Image` component that displays the skill icon.
- **`outlineT`** – Transform controlling the node's selection outline.
- **`maxedOutlineT`** – Transform shown when the node is fully upgraded.
- **`button`** – The UI button that allows the player to interact with the node.
- **`AvaibleColor`** – Color applied when the node is available to purchase.
- **`LockedColor`** – Color applied when the node is locked.
- **`UnlockedColor`** – Color applied when the node is unlocked but not yet maxed.
- **`lineColor`** / **`lineUnlockedColor`** – Colors used to visually differentiate connected lines depending on the node's state.

Methods

- **`Initialize(SkillNode skillNode)`**
  Links the visual component to its parent `SkillNode` logic instance.
- **`Selected()` / `Deselected()`**
  Toggles the visibility of the selection outline when the player selects or deselects a node.
- **`BuyableVisual()` / `LockedVisual()` / `UnlockedVisual()` / `MaxedVisual()`**
  Updates the node's appearance depending on its current gameplay state:
  - **BuyableVisual:** Highlights node as available for purchase.
  - **LockedVisual:** Dims the icon to indicate it is locked.
  - **UnlockedVisual:** Restores normal color after unlocking.
  - **MaxedVisual:** Adds a distinct outline for fully upgraded nodes.
- **`UpdateLineColor()`**
  Updates all connected `SkillNodeLine` colors based on whether the node is unlocked or not.

Skill Tooltip UI

The `SkillTooltipUI` class manages the **tooltip window** that displays detailed information about a skill node when selected.
It provides real-time updates on skill name, cost, level, description, and unlock requirements, ensuring the player always sees accurate information about the selected skill.

- Displays dynamic information about a selected skill node.
- Updates text fields (name, cost, level, description, and requirements).
- Adapts its position on screen to avoid clipping at the edges.
- Hides automatically when no skill node is selected.

Properties

- **`controller`** – Reference to the `SkillTreeController`, used to access skill point data (e.g., currency name).
- **`window`** – The root `GameObject` of the tooltip window; shown or hidden as needed.
- **`titleTMP`** / **`descriptionTMP`** / **`requiredTMP`** / **`costTMP`** / **`levelTMP`** – Text elements displaying each piece of skill information.
- **`priceContainer`** / **`requiredContainer`** – UI containers used to toggle visibility depending on whether the node is locked or unlocked.
- **`rect`** – Cached `RectTransform` used to position the tooltip correctly relative to the mouse.
- **`node`** – The currently selected `SkillNode` whose data is being displayed.

Methods

- **`Show(SkillNode node)`**
  Displays the tooltip and updates it with the selected node's data.
- **`Hide()`**
  Hides the tooltip window and clears the current reference.
- **`UpdateInfo()`**
  Refreshes all tooltip text fields based on the current `SkillNode` state:
  - Shows the **skill name**, **description**, and **current level**.
  - Displays the **cost** and colors it red if the player cannot afford it.
  - Shows **required parent skills** if the node is still locked.
- **`Update()`**
  Continuously updates the tooltip position each frame, ensuring it follows the node and remains visible on-screen.
- **`ResolveCorner()`**
  Adjusts the tooltip pivot dynamically based on the mouse position to prevent the window from going off-screen horizontally.